

Security Audit

Honeypot Finance 2nd (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	22
Conclusion	23
Our Methodology	24
Disclaimers	26
About Hashlock	27

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Honeypot team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Honeypot Finance is a decentralized finance (DeFi) protocol focused on creating sustainable yield opportunities through structured investment products. Built on Ethereum, the platform offers users access to various vaults and strategies designed to optimize returns while managing risk. Its core value proposition revolves around transparency, community governance, and composability, allowing users to participate in automated strategies without giving up control of their assets. Honeypot aims to balance innovation and security, enabling more predictable and responsible DeFi participation.

Project Name: Honeypot Finance

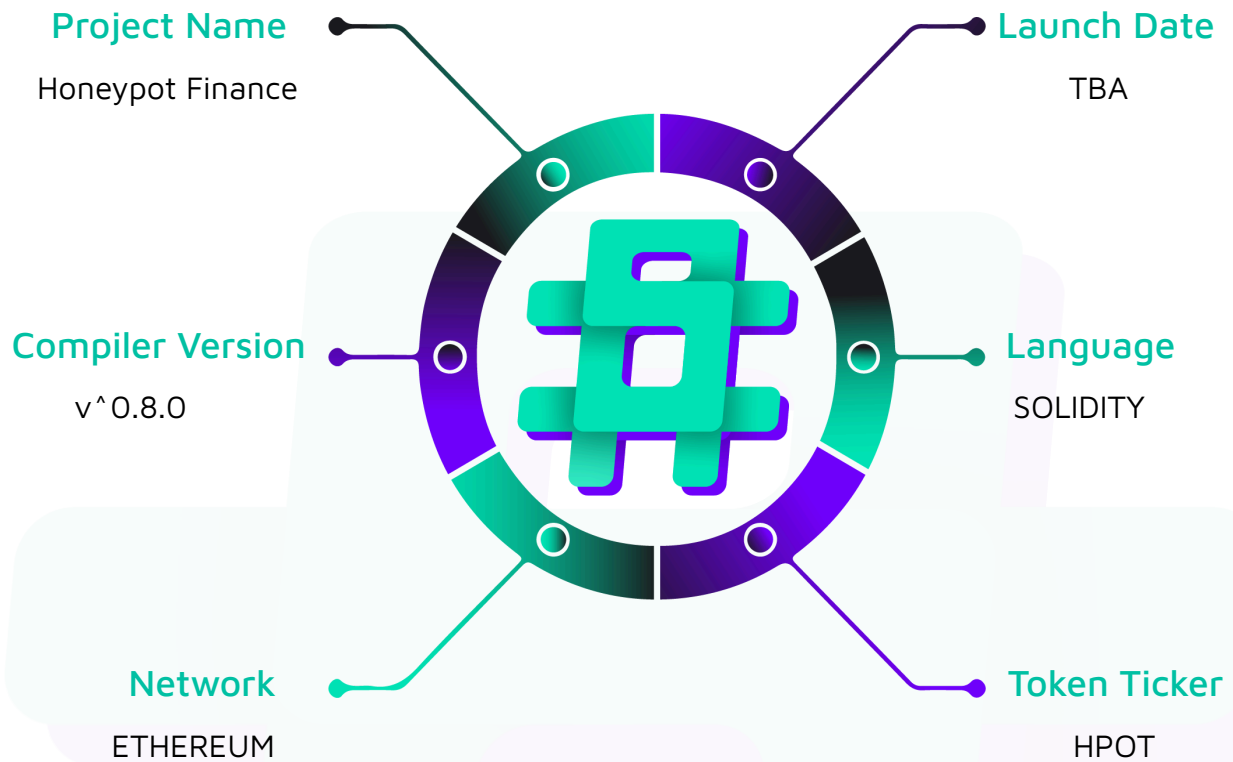
Project Type: DeFi, Vault, Token

Compiler Version: ^0.8.0

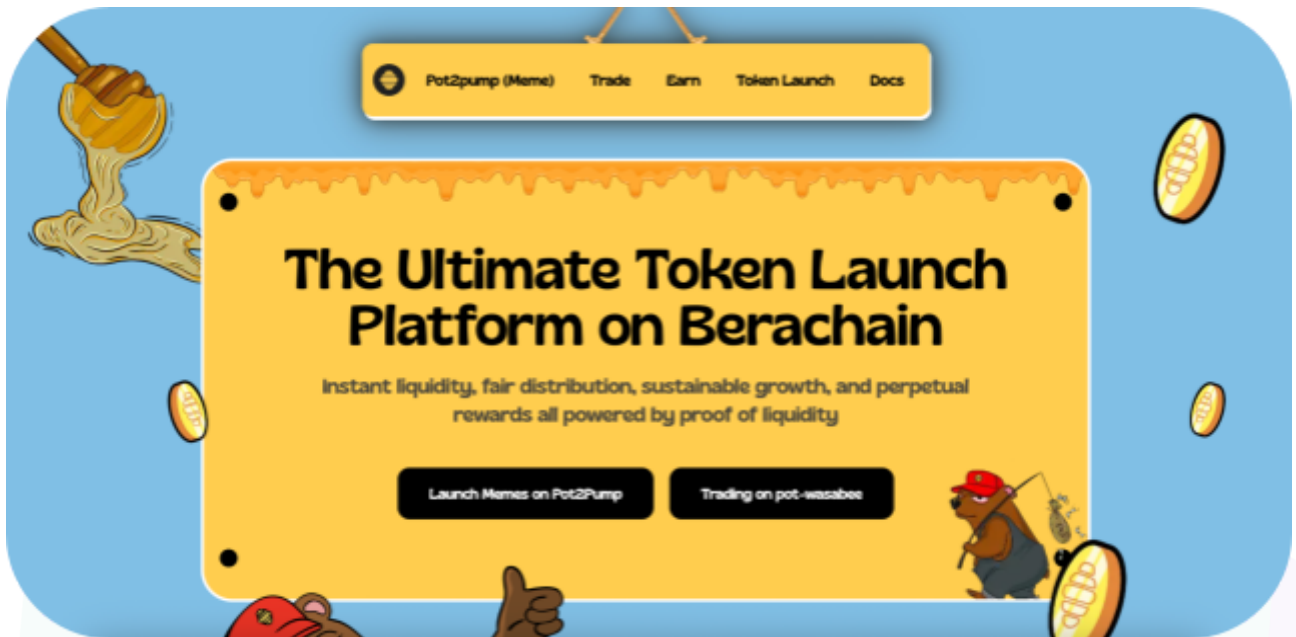
Website: <https://honeypotfinance.xyz/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Honeypot Finance project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Honeypot Finance Smart Contracts
Platform	Ethereum / Solidity
Audit Date	May, 2025
Contract 1	AllInOneVault.sol
Contract 1 MD5 Hash	fb33b4d5caa3002cd0c2163b46571ee2
Contract 2	TxReceipt.sol
Contract 2 MD5 Hash	e70584278b3242344b3f7ed401a222e6
Audited GitHub Commit Hash	d89585bb3059d4b65f12cf63caa79a480739d99c
Fix Review GitHub Commit Hash	256c0b8838a18e8a139a4110849a92dd6117a9ec

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium severity vulnerabilities

2 Low severity vulnerabilities

3 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
AllInOneVault.sol - Allows users to: - Burn supported tokens to receive receipts - Claim IBGT rewards after cooldown period - Trigger reward minting from BeraPawForge - Check receipt status and eligibility - Allows admins to: - Add and update supported tokens with weights - Remove tokens from supported list - Set cooldown time for claiming rewards - Unstake tokens in emergency situations - Upgrade contract implementation	Contract achieves this functionality.
TxReceipt.sol - Allow users to: - Claim token airdrops via Merkle proof verification - Check if they've already claimed for a round - View current airdrop round information - Transfer and use the ERC20 token normally - Allow admins to: - Set Merkle roots for airdrop verification - Start new airdrop rounds - Manage the overall token distribution process - Retain control of token contract ownership	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Honeypot Finance project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Honeypot Finance project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] AllInOneVault#claim - Inconsistent accounting of claim functionality can result in user getting more rewards than intended.

Description

The current claim functionality is based on users getting a receipt of a token to claim LBGT after the cooldown period is over. However, after each successive claim, `totalWeight` is reduced causing the next claim to be calculated against a small denominator. Since the rewards will grow every time `claimBGT2Rewards` is called, users who claim last can gain significantly more tokens than others.

Vulnerability Details

Consider the following flow. Whenever, `getReceipt` is called, `totalWeight` increases based on the token address provided. Whenever, claim is called, `totalWeight` decreases and LBGT rewards increase.

Consider the scenario where 3 users deposited 1000 units of TokenA with 1:1 weight. After 7 days, all of them called claim.

```
function claim(uint256 receiptID) external override {  
    Receipt storage receipt = receipts[receiptID];  
    require(receipt.user == msg.sender, "Not the owner of the receipt");  
    require(block.timestamp >= receipt.claimableAt, "Not claimable yet");  
    require(!receipt.claimed, "Reward already claimed");  
  
    claimBGT2Rewards();  
    uint256 poolReward = IERC20(LBGTAddress).balanceOf(address(this));  
  
    require(poolReward > 0, "No pool rewards to claim");  
}
```

```

uint256 reward = (receipt.receiptWeight * poolReward) / totalWeight;
require(reward > 0, "No rewards to claim");

// 先更新状态
receipt.claimed = true;
totalWeight -= receipt.receiptWeight;

// 最后进行转账
IERC20(lBGTAddress).safeTransfer(msg.sender, reward);

emit Claimed(msg.sender, receiptID, reward, block.timestamp);
}

```

Mathematically according to the tests, rewards are generated when user claims. Upon each claim, there are more rewards in the contract overall because it triggers mint.

Proof of Concept

Paste for following POC in AllInOneVault.t.sol

```

function test_RewardDistributionBug() public {
    // Setup 3 users with equal stakes
    address user1 = makeAddr("user1");
    address user2 = makeAddr("user2");
    address user3 = makeAddr("user3");

    // Transfer tokens to all users
    token.transfer(user1, 1000 * 10 ** token.decimals());
    token.transfer(user2, 1000 * 10 ** token.decimals());
    token.transfer(user3, 1000 * 10 ** token.decimals());

    // Admin sets up token weight
    vm.startPrank(admin);
    vault.updateSupportedToken(address(token), 10000); // 1:1 weight
    vm.stopPrank();

    // Each user gets a receipt for the same amount
}

```

```
uint256 stakeAmount = 1000;

// User 1 gets receipt
vm.startPrank(user1);
token.approve(address(vault), stakeAmount);
vault.getReceipt(address(token), stakeAmount);
vm.stopPrank();

// User 2 gets receipt
vm.startPrank(user2);
token.approve(address(vault), stakeAmount);
vault.getReceipt(address(token), stakeAmount);
vm.stopPrank();

// User 3 gets receipt
vm.startPrank(user3);
token.approve(address(vault), stakeAmount);
vault.getReceipt(address(token), stakeAmount);
vm.stopPrank();

// Fast forward time past cooldown
vm.warp(block.timestamp + vault.cooldownTime() + 1);

// Record initial balances
uint256 user1InitialBalance = LBGT.balanceOf(user1);
uint256 user2InitialBalance = LBGT.balanceOf(user2);
uint256 user3InitialBalance = LBGT.balanceOf(user3);

// User 1 claims first
vm.startPrank(user1);
vault.claim(0);
vm.stopPrank();

// User 2 claims second
vm.startPrank(user2);
vault.claim(1);
vm.stopPrank();
```

```

// User 3 claims last
vm.startPrank(user3);
vault.claim(2);
vm.stopPrank();

// Calculate rewards received
uint256 user1Reward = IBGT.balanceOf(user1) - user1InitialBalance;
uint256 user2Reward = IBGT.balanceOf(user2) - user2InitialBalance;
uint256 user3Reward = IBGT.balanceOf(user3) - user3InitialBalance;

// Log the results
console.log("User 1 reward:", user1Reward);
console.log("User 2 reward:", user2Reward);
console.log("User 3 reward:", user3Reward);
}

```

When User 1 claims:

First mint occurs: 1,000,000 tokens

User 1 gets 1/3 of these tokens: 333,333 tokens

When User 2 claims:

Second mint occurs: another 1,000,000 tokens

Total pool now has 1,666,667 tokens (new + leftover)

User 2 gets 2/3 of their share: 666,666 tokens

When User 3 claims:

Third mint occurs: another 1,000,000 tokens

Total pool now has 2,000,000 tokens (new + leftover)

User 3 gets 3/3 of their share: 999,999 tokens.

Based on the results, we can see that the last claimer got significantly more amount than the rest of them.

[PASS] test_RewardDistributionBug() (gas: 750838)

Logs:


```
User 1 reward: 33333333333333333333333333333333
User 2 reward: 83333333333333333333333333333333
User 3 reward: 18333333333333333333333333333334
```

Impact

The reward distribution can be gamed in a way where the last claimers are getting more rewards than others.

Recommendation

Implement a `rewardPerShare` mechanism. Track a global `rewardPerShareStored` value (typically scaled by `1e18` for precision). For each receipt, track the last `rewardPerSharePaid` value when they claimed. When claiming, users receive their proportional share of all rewards accumulated since their last claim.

Each time new rewards are minted, the `rewardPerShareStored` increases equally for all users. When User A claims, they receive exactly their proportion of all accumulated rewards followed by UserB and UserC.

Keep in mind that initially : Later claimants got more rewards due to a mathematical error (decreasing totalWeight).

After fix : Later claimants should get more rewards because more rewards are being minted when they claim.

Status

Acknowledged

Low

[L-01] TxReceipt - Use Ownable2Step.sol instead

Description

The contract uses the single-step Ownable pattern for ownership management, which transfers ownership immediately upon calling `transferOwnership()`. This creates a risk that ownership could be accidentally transferred to an incorrect or inaccessible address, permanently locking admin functions.

Impact

Risk of permanently losing contract ownership if transferred to an incorrect address

Recommendation

Replace the current Ownable implementation with OpenZeppelin's Ownable2Step, which implements a two-step ownership transfer

Status

Resolved

[L-02] TxReceipt - setMerkleRoot should check if current round is already set**Description**

The current implementation allows the owner to call setMerkleRoot() multiple times for the same airdrop round, overwriting the previously set merkle root without any restrictions.

Impact

If the merkle root is changed after some users have claimed, the new root could include users who've already claimed hence overwriting existing merkle root by admin is possible.

Recommendation

Add a check that prevents overwriting an existing merkle root

Status

Resolved

QA

[Q-01] AllInOneVault - Contract logic does not account for weird tokens .

Description

The current implementation of vault assumes tokens to be of the same decimals and are not fee-on-transfer type.

This invariant can be broken if tokens like usdt (6 decimals) are used here.

Recommendation

Ensure that only standard ERC20 is allowed. If non-standard ERC20 is allowed, change the code accordingly.

Status

Acknowledged

[Q-02] AllInOneVault - Unused Variable

Description

The AllInOneVault contract declares a state variable address public receiptToken that is never initialized, used, or referenced in any contract logic. This variable consumes storage space and increases deployment gas costs without providing any functionality to the contract.

Recommendation

Remove the unused variable to improve code quality and reduce deployment costs.

Status

Resolved

[Q-03] TxReceipt - Emit event for critical state changes

Description

The `startNewAirdropRound()` function modifies a critical state variable (`currentAirdropRound`) but does not emit an event to log this change.

Recommendation

Emit event for state changes.

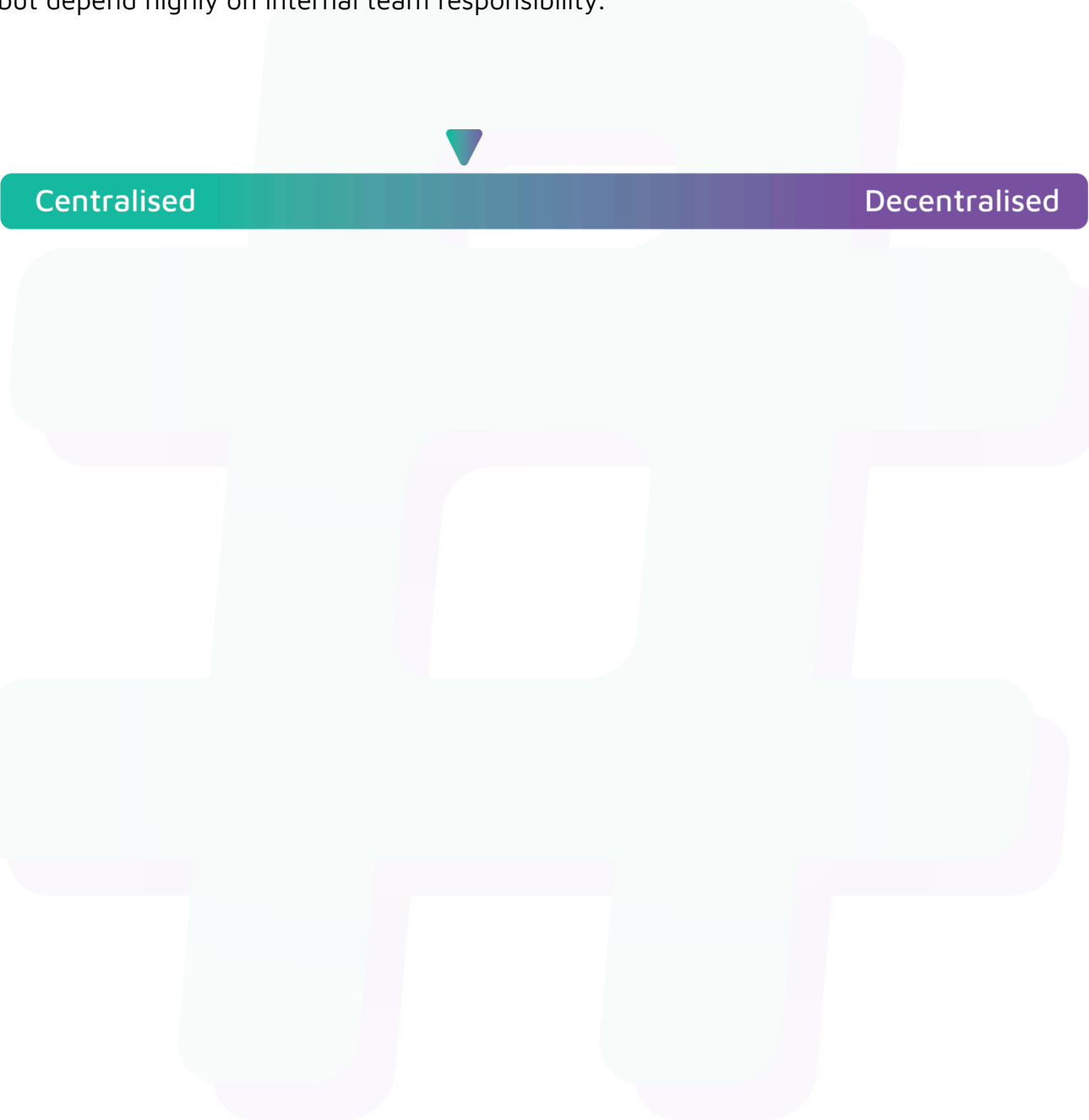
Status

Resolved

Centralisation

The Honeypot Finance project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Honeypot Finance project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.